

UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Facultad de Ingeniería de Sistemas e Informática



“Segundo Entregable”

CURSO

Estructura de Datos

DOCENTE

Arredondo Castillo, Gustavo

INTEGRANTES GRUPO 6

Cerna Sifuentes, Augusto Nicolas	24200053
Coronado Lucano, Xiomara Elizabeth	24200223
Siesquen Torres, Katherine Lizbeth	24200066
Postigo Vega, Diana Carolina	24200167
Grijalva De La Cruz, Rony Smith	24200058

Lima, Perú
2025

Índice

a. Especificación de entradas, salidas y estructuras por funcionalidad.....	3
b. Correspondencia entre estructuras de datos y funcionalidades del sistema.....	5
c. Progreso de implementación por integrante y módulo.....	6
d. Interfaz inicial: pantalla principal y menú de opciones.....	18

a. Especificación de entradas, salidas y estructuras por funcionalidad

Estructura usada	Input (Entrada)	Funcionalidad dentro del sistema	Resultado esperado
Lista Enlazada y Algoritmos de Ordenamiento (Mergesort)	1. Lista de conciertos: Colección completa de eventos cargados desde la base de datos al iniciar la aplicación. 2. Criterio del usuario: Filtro seleccionado en la interfaz (ej: "precio", "fecha" o "popularidad").	Gestiona la colección de conciertos en memoria de forma eficiente. Permite reordenar la lista dinámicamente según la preferencia del usuario sin necesidad de consultar la base de datos repetidamente.	Se muestra al usuario una lista de conciertos en la página de inicio, ordenada según el criterio que seleccionó.

Módulo Usuario:

Estructura usada	Input (código / QR)	Funcionalidad dentro del sistema	Resultado esperado
Diccionario (Usuarios)	Username y contraseña	Iniciar Sesion()	Retorna el username y el rol del Usuario si las credenciales son válidas. En caso contrario, retorna un mensaje de error.
Diccionario (Usuarios) Conjunto (dni_registrados) Conjunto (usernames_registrados)	Datos completos del usuario (Nombre, DNI, Email, Contraseña, etc.)	Registrar Usuario ()	Se verifica que dni, username y correo sean únicos. El nuevo Usuario se añade al diccionario Usuarios. Retorna un mensaje de confirmación o el objeto creado.

Archivo CSV (usuarios.xlsx)	Nombre del archivo	cargar_usuarios_de_sde_excel()	El diccionario Usuarios del sistema se llena con todos los Objetos Usuario leídos del archivo.
--------------------------------	--------------------	--------------------------------	---

Módulo para validación y control de acceso:

Estructura usada	Input (código / QR)	Funcionalidad dentro del sistema	Resultado esperado
Cola (Queue)	Lista de códigos de entrada (QR o numéricos) que llegan desde el lector o formulario web. Ejemplo: ["QT1001", "QT1002", "QT1003"].	Gestiona el orden de llegada de los tickets al sistema. Los códigos se encolan y se procesan de forma secuencial (FIFO).	Se valida primero QT1001 , luego QT1002 y por último QT1003 . Se mantiene un flujo ordenado y controlado.
Conjunto (Set)	Conjunto de códigos ya validados: { "QT1002", "QT1010" } y nuevo ingreso "QT1002".	Comprueba si el código que ingresa ya fue validado previamente. Evita duplicidades en el acceso al evento.	El sistema detecta duplicado y muestra: "Código ya utilizado". No se permite el ingreso.
Árbol Binario (BST)	Códigos de entradas validadas ordenadas: ["QT1001", "QT1003", "QT1002"].	Almacena los códigos QR validados en forma jerárquica. Permite búsquedas rápidas para confirmar la validez de un código.	Si el código "QT1003" existe en el árbol → "Ticket válido". Si no, "Ticket inválido".

Modulo: Carrito

Estructura usada	Input (Entrada)	Funcionalidad dentro del sistema	Resultado esperado
------------------	-----------------	----------------------------------	--------------------

Lista de ítems del carrito	Evento seleccionado, zona, cantidad	Agregar entrada al carrito	Ítem agregado, totales actualizados y confirmación visual.
Lista de ítems del carrito	Ítem del carrito, nueva cantidad o acción “quitar”	Ajustar cantidad / quitar ítem	Totales recalculados; si se quita, el ítem desaparece.
Pila de acciones (historial)	Comando “Deshacer”	Revertir la última acción realizada en el carrito	Estado del carrito vuelve al paso anterior; totales se actualizan.
Lista de ítems del carrito	(acción desde la vista del carrito)	Mostrar resumen previo a pago	Desglose de ítems y total; opción de continuar o editar.

b. Correspondencia entre estructuras de datos y funcionalidades del sistema

Módulos	Estructura que usa	Funcionalidades
Catálogo de eventos	Lista enlazada + ordenamiento/búsqueda	Portada/lista de conciertos
Zonas y asientos	Matriz + cola circular	Detalle del evento y reservas
Usuarios (registro/login)	Diccionario + Conjuntos + Listas + Archivo CSV	Registrar_usuario(), iniciar_sesion(), agregar_ticket_a_usuario(), obtener_compras(),

		obtener_info_usuario(), cargar_usuarios_desde_excel(), actualizar_excel()
Carrito con “Deshacer”	Pila	Carrito, revertir última acción
Validación de tickets	Cola + conjunto/árbol	Puerta de acceso (staff)

c. Progreso de implementación por integrante y módulo

1) Módulo de Catálogo de eventos:

El avance se centra en la correcta aplicación de las estructuras de datos solicitadas para gestionar y presentar los conciertos de manera dinámica.

1. Creación del Modelo de Datos (models.py)

Se definió la estructura Evento en Django, que sirve como molde para almacenar la información persistente de los conciertos en una base de datos. Esto permite que el sistema guarde y recupere datos como nombre, fecha, precio y popularidad.

```
from django.db import models

class Evento(models.Model):
    nombre = models.CharField(max_length=200)
    artista = models.CharField(max_length=200)
    fecha = models.DateTimeField()
    lugar = models.CharField(max_length=200)
    precio = models.DecimalField(max_digits=10, decimal_places=2)
    popularidad = models.IntegerField(default=0)

    def __str__(self):
        return f"{self.nombre} - {self.artista}"

    class Meta:
        ordering = ['-fecha']
```

2. Implementación de la Lista Enlazada (structures.py)

Este es el núcleo del avance. Se desarrolló una clase `ListaEnlazadaEventos` que maneja los datos en memoria. La funcionalidad más importante implementada es el método de ordenamiento `ordenar_por(criterio)`, que utiliza el algoritmo Mergesort. Se eligió Mergesort por su eficiencia y su buena adaptación al trabajo con listas enlazadas, ya que no requiere acceso aleatorio a los elementos. El código lee los eventos de la base de datos, los inserta en la lista enlazada y los reordena eficientemente según el criterio solicitado por el usuario (precio, fecha o popularidad).

```
from datetime import datetime
from decimal import Decimal
from typing import Any, Optional, List, Union

class Nodo:
    def __init__(self, evento: dict):
        self.evento = evento
        self.siguiente: Optional[Nodo] = None

class ListaEnlazadaEventos:
    def __init__(self):
        self.cabeza: Optional[Nodo] = None

    def agregar_evento(self, evento_simple: dict) -> None:
        nuevo_nodo = Nodo(evento_simple)
        nuevo_nodo.siguiente = self.cabeza
        self.cabeza = nuevo_nodo

    def a_lista_simple(self) -> List[dict]:
        eventos = []
        actual = self.cabeza
        while actual:
            eventos.append(actual.evento)
            actual = actual.siguiente
        return eventos

    def _get_middle(self, head: Optional[Nodo]) -> Optional[Nodo]:
        if not head:
            return None

        tortuga = head
        liebre = head.siguiente
```

```

class ListaEnlazadaEventos:
    def _get_middle(self, head: Optional[Nodo]) -> Optional[Nodo]:

        while liebre and liebre.siguiente:
            tortuga = tortuga.siguiente
            liebre = liebre.siguiente.siguiente

        return tortuga

    def _sorted_merge(self, izq: Optional[Nodo], der: Optional[Nodo], criterio: str) -> Optional[Nodo]:
        if not izq:
            return der
        if not der:
            return izq

        def comparar(a: dict, b: dict, campo: str) -> bool:
            if campo == 'precio':
                return Decimal(str(a[campo])) <= Decimal(str(b[campo]))
            elif campo == 'fecha':
                fecha_a = datetime.fromisoformat(a[campo]) if isinstance(a[campo], str) else a[campo]
                fecha_b = datetime.fromisoformat(b[campo]) if isinstance(b[campo], str) else b[campo]
                return fecha_a <= fecha_b
            else:
                return a[campo] <= b[campo]

        if comparar(izq.evento, der.evento, criterio):
            resultado = izq
            resultado.siguiente = self._sorted_merge(izq.siguiente, der, criterio)
        else:
            resultado = der
            resultado.siguiente = self._sorted_merge(izq, der.siguiente, criterio)

```

```

        return resultado

    def _mergesort(self, head: Optional[Nodo], criterio: str) -> Optional[Nodo]:
        if not head or not head.siguiente:
            return head

        medio = self._get_middle(head)
        if not medio:
            return head

        segunda_mitad = medio.siguiente
        medio.siguiente = None

        izq = self._mergesort(head, criterio)
        der = self._mergesort(segunda_mitad, criterio)

        return self._sorted_merge(izq, der, criterio)

    def ordenar_por(self, criterio: str) -> None:
        if criterio not in ['precio', 'fecha', 'popularidad']:
            raise ValueError("Criterio de ordenamiento no válido")

        self.cabeza = self._mergesort(self.cabeza, criterio)

```

3. Desarrollo de la Vista API (views.py y urls.py)

Finalmente, se creó un endpoint en la API para que el frontend (la página web) pueda solicitar la lista de eventos. La vista se encarga de:

- Recibir la petición del usuario.
- Cargar los datos desde la base de datos.
- Utilizar la ListaEnlazadaEventos para ordenarlos en memoria.
- Devolver la lista ya ordenada en formato JSON, lista para ser mostrada en la página.

Esto cumple con la directiva del curso de usar la base de datos solo para almacenamiento, mientras que la manipulación principal de los datos se realiza con estructuras dinámicas.

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import Evento
from .structures import ListaEnlazadaEventos

@api_view(['GET'])
def catalogo_de_eventos(request):
    criterio = request.query_params.get('ordenar', 'fecha')

    if criterio not in ['precio', 'fecha', 'popularidad']:
        return Response(
            {"error": "Criterio de ordenamiento no válido"},
            status=400
        )

    eventos = Evento.objects.all()
    lista_enlazada = ListaEnlazadaEventos()

    for evento in eventos:
        evento_dict = {
            'id': evento.id,
            'nombre': evento.nombre,
            'artista': evento.artista,
            'fecha': evento.fecha.isoformat(),
            'lugar': evento.lugar,
            'precio': str(evento.precio),
            'popularidad': evento.popularidad
        }
        lista_enlazada.agregar_evento(evento_dict)

    lista_enlazada.ordenar_por(criterio)
    eventos_ordenados = lista_enlazada.a_lista_simple()
```

```

from django.contrib import admin
from django.urls import path
from events.views import catalogo_de_eventos

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/eventos/', catalogo_de_eventos, name='catalogo-eventos'),
]

```

3) Módulo Usuarios

__init__(self): Inicializa el sistema creando las estructuras de datos principales: un diccionario para almacenar los usuarios por email, dos conjuntos para rastrear DNI y nombres de usuario únicos, y un contador para generar IDs. También inicia la carga de datos desde el Excel.

```

class SistemaUsuarios:
    def __init__(self):

        self.usuarios = {} #Se usa un diccionario con email como clave para acceso
        self.dni_registrados = set() #Conjunto para verificar DNIs únicos
        self.usernames_registrados = set() #Conjunto para verificar nombres

        self.contadorID = 1

        self.excel_file = 'usuarios.xlsx'#Archivo excel donde se almacenan los usuarios
        self.cargar_usuarios_desde_excel()#al incializar el sistema, cargar usuarios existentes

```

cargar_usuarios_desde_excel(self): Lee y procesa el archivo usuarios.xlsx. Esta función reconstruye los usuarios en memoria y actualiza los conjuntos de DNI y usernames para que el sistema sepa qué identificadores están ya en uso, asegurando que el contador de IDs no cree duplicados.

```

def cargar_usuarios_desde_excel(self):
    if os.path.exists(self.excel_file):
        df = pd.read_excel(self.excel_file)
        for _, row in df.iterrows():
            usuario = {
                'id': row['id'],
                'username': row['username'],
                'contraseña': row['contraseña'],
                'email': row['email'],
                'nombre': row['nombre'],
                'apellido': row['apellido'],
                'edad': row['edad'],
                'dni': row['dni'],
                'telefono': row['telefono'],
                'rol': row['rol'],
                'tickets_comprados': eval(row['tickets_comprados']) if pd.notna(row['tickets_comprados']) else []
            }
            self.usuarios[usuario['email']] = usuario
            self.dni_registrados.add(usuario['dni'])
            self.usernames_registrados.add(usuario['username'])
            if usuario['id'] >= self.contadorID:
                self.contadorID = usuario['id'] + 1

```

_actualizar_excel(self): Es la función interna que guarda el estado actual de todos los usuarios en el diccionario de vuelta al archivo usuarios.xlsx. Esto asegura que los cambios (como un nuevo registro o la compra de un ticket) persistan.

```
def _actualizar_excel(self):
    data = [] #Lista para almacenar datos de usuarios
    for usuario in self.usuarios.values():
        data.append({
            'id': usuario['id'],
            'username': usuario['username'],
            'contraseña': usuario['contraseña'],
            'email': usuario['email'],
            'nombre': usuario['nombre'],
            'apellido': usuario['apellido'],
            'edad': usuario['edad'],
            'dni': usuario['dni'],
            'telefono': usuario['telefono'],
            'rol': usuario['rol'],
            'tickets_comprados': str(usuario['tickets_comprados'])
        })
    df = pd.DataFrame(data)
    df.to_excel(self.excel_file, index=False)
```

registrar_usuario(...): Se encarga de crear una nueva cuenta. Valida rigurosamente que todos los campos estén completos, que la contraseña cumpla el mínimo de 8 caracteres, que el email, DNI y username sean únicos, y que la edad esté en el rango de 18 a 100 años. Si todas las validaciones pasan, añade el usuario al sistema y actualiza el Excel.

```
def registrar_usuario(self, username, contraseña, email, nombre, apellido, edad, dni, telefono):

    # Validar que todos los campos estén completos
    if not all([username, contraseña, email, nombre, apellido, edad, dni, telefono]):
        return (False, "Todos los campos son obligatorios.")

    # Validar que la contraseña tenga mas de 8 caracteres
    if len(contraseña) < 8:
        return (False, "La contraseña debe tener al menos 8 caracteres.")

    # Validar el formato del correo
    if not re.match(r"^[a-zA-Z0-9]+@[a-zA-Z0-9]+\.[a-zA-Z]{2,}$", email):
        return (False, "Formato de email invalido.")

    # Validar formato del DNI
    if len(str(dni)) != 8 or not str(dni).isdigit():
        return (False, "DNI Invalido (debe tener 8 digitos).")

    # Validar rango de edad
    try:
        edad_num = int(edad)
        if edad_num < 18 or edad_num > 100:
            return (False, "Edad fuera del rango (18-100 años).")
    except ValueError:
        return (False, "La edad debe ser un numero.")

    # Validar que email y username sean únicos
    if email in self.usuarios:
        return (False, "Email ya registrado.")

    if username in self.usernames_registrados:
        return (False, "El nombre de usuario ya esta en uso.")

    if dni in self.dni_registrados:
        return (False, "DNI ya registrado.")

    # Crear ID único
    usuarioID = self.contadorID
    self.contadorID += 1
```

```
#Creacion de nuevo usuario
nuevo_usuario = {
    'id': usuarioID,
    'username': username,
    'contraseña': contraseña,
    'email': email,
    'nombre': nombre,
    'apellido': apellido,
    'edad': edad_num,
    'dni': dni,
    'telefono': telefono,
    'rol': 'usuario',
    'tickets_comprados': []
}

# Almacenar el nuevo usuario en diccionario y registrar nuevo dni y username en conjuntos
self.usuarios[email] = nuevo_usuario
self.dni_registrados.add(dni)
self.usernames_registrados.add(username)

self.actualizar_excel() #Actualizar archivo excel con nuevo usuario
return (True, "Usuario registrado con éxito.")
```

iniciar_sesion(self, email, contraseña): Intenta autenticar a un usuario. Verifica si el email existe y luego compara la contraseña proporcionada. Devuelve un estado de éxito/fracaso, el mensaje correspondiente, y el username y rol del usuario autenticado.

```
def iniciar_sesion(self, email, contraseña):

    usuario = self.usuarios.get(email)

    if not usuario:
        return (False, "Email no registrado.")

    if str(usuario['contraseña']) == str(contraseña):
        return (True, "Inicio de sesion exitoso.", usuario['username'], usuario['rol'])
    else:
        return (False, "Contraseña incorrecta.", None, None)
```

agregar_ticket_a_usuario(self, email, ticket_id): Permite asociar la compra de un ticket específico (identificado por ticket_id) a la cuenta del usuario. Una vez agregado a la lista de 'tickets_comprados', guarda la modificación en el archivo Excel.

```
def agregar_ticket_a_usuario(self, email, ticket_id):
    usuario = self.usuarios.get(email)
    if usuario:
        usuario['tickets_comprados'].append(ticket_id)
        self._actualizar_excel()
        return (True, "Ticket agregado a la cuenta.")
    return (False, "Usuario no encontrado.")
```

obtener_compras_usuario(self, email): Busca al usuario por email y devuelve su lista completa de IDs de tickets comprados.

```
def obtener_compras_usuario(self, email):
    usuario = self.usuarios.get(email)
    if usuario:
        return (True, usuario['tickets_comprados'])
    return (False, "Usuario no encontrado.")
```

obtener_info_usuario(self, email): Recupera todos los datos del perfil de un usuario. Para proteger la privacidad, devuelve una copia de la información omitiendo la contraseña.

```
def obtener_info_usuario(self, email):
    usuario = self.usuarios.get(email)
    if usuario:
        info_perfil = usuario.copy()
        info_perfil.pop('contraseña', None)
        return (True, info_perfil)
    return (False, "Usuario no encontrado.")
```

5. Módulo para validación y control de acceso:

5.1 Creación del módulo y modelos en Django

El desarrollo inició con la creación de la aplicación **validation** dentro del backend Django.

En el archivo **models.py** se definieron las clases **Entrada** y **Validacion**, que representan las entradas registradas por los clientes y los resultados de cada proceso de validación.

Cada validación guarda su relación con la entrada correspondiente y registra la fecha, el resultado y el estado del código (válido, usado o inválido).

```

Zentry > modules > validation > models.py > ...
1  from django.db import models
2  from django.utils import timezone
3
4  class Entrada(models.Model):
5      codigo_qr = models.CharField(max_length=50, unique=True)
6      estado = models.CharField(max_length=20, default='valido')
7      id_compra = models.IntegerField()
8
9      def __str__(self):
10         return self.codigo_qr
11
12 class Validacion(models.Model):
13     entrada = models.ForeignKey(Entrada, on_delete=models.CASCADE, null=True)
14     fecha_validacion = models.DateTimeField(default=timezone.now)
15     resultado = models.CharField(max_length=20)
16
17     def __str__(self):
18         return f"{self.resultado} - {self.fecha_validacion}"
19

```

5.2 Implementación de estructuras de datos (structures.py)

Para cumplir con los objetivos del curso de **Estructura de Datos**, se diseñó el archivo `structures.py`, donde se implementaron tres estructuras personalizadas:

1. **Cola (Queue)** → controla el orden de llegada de los códigos al sistema.
2. **Conjunto (Set)** → detecta y evita validaciones duplicadas.
3. **Árbol Binario (BST)** → organiza y busca códigos validados de manera eficiente.

```

Zentry > modules > validation > structures.py > ...
1  # --- Cola ---
2  class Cola:
3      def __init__(self):
4          self.items = []
5
6      def enqueue(self, item):
7          self.items.append(item)
8
9      def dequeue(self):
10         if not self.estaVacia():
11             return self.items.pop(0)
12
13     def estaVacia(self):
14         return len(self.items) == 0
15
16 # --- Conjunto ---
17 class SetEntradas:
18     def __init__(self):
19         self.usados = set()
20
21     def agregar(self, codigo):
22         self.usados.add(codigo)
23
24     def existe(self, codigo):
25         return codigo in self.usados
26
27 # --- Árbol Binario ---
28 class Nodo:
29     def __init__(self, codigo):
30         self.codigo = codigo
31         self.izq = None
32         self.der = None
33
34 class ArbolQR:
35     def __init__(self):
36         self.raiz = None
37

```

```

38     def insertar(self, codigo):
39         self.raiz = self._insertar_rec(self.raiz, codigo)
40
41     def _insertar_rec(self, nodo, codigo):
42         if nodo is None:
43             return Nodo(codigo)
44         if codigo < nodo.codigo:
45             nodo.izq = self._insertar_rec(nodo.izq, codigo)
46         else:
47             nodo.der = self._insertar_rec(nodo.der, codigo)
48         return nodo
49
50     def buscar(self, codigo):
51         return self._buscar_rec(self.raiz, codigo)
52
53     def _buscar_rec(self, nodo, codigo):
54         if nodo is None:
55             return False
56         if nodo.codigo == codigo:
57             return True
58         elif codigo < nodo.codigo:
59             return self._buscar_rec(nodo.izq, codigo)
60         else:
61             return self._buscar_rec(nodo.der, codigo)

```

Estas estructuras son utilizadas internamente por la vista principal del módulo (**views.py**) cada vez que el staff realiza una validación.

5.3 Desarrollo de la vista principal y conexión con las estructuras

El archivo **views.py** implementa el endpoint **/api/validar/**, el cual recibe los códigos enviados desde el frontend (ya sea escritos manualmente o escaneados desde un QR).

El código recibido se agrega a la **cola**, se verifica en el **set** y finalmente se busca en el **árbol** para determinar su validez antes de actualizar la base de datos

```
Zentry > modules > validation > views.py > validar_ticket
1  from rest_framework.decorators import api_view
2  from rest_framework.response import Response
3  from .models import Entrada, Validacion
4  from .structures import Cola, SetEntradas, ArbolQR
5
6  cola_validacion = Cola()
7  set_usados = SetEntradas()
8  arbol_codigos = ArbolQR()
9
10 @api_view(['POST'])
11 def validar_ticket(request):
12     codigo = request.data.get('codigo')
13
14     cola_validacion.enqueue(codigo) # Se agrega a la cola
15
16     if set_usados.existe(codigo):
17         Validacion.objects.create(entrada=None, resultado='Usado')
18         return Response({'mensaje': ' Código ya utilizado'}, status=400)
19
20     try:
21         entrada = Entrada.objects.get(codigo_qr=codigo)
22     except Entrada.DoesNotExist:
23         Validacion.objects.create(entrada=None, resultado='Inválido')
24         return Response({'mensaje': ' Código inválido'}, status=404)
25
26     # Se marca como usado y se agrega al conjunto y al árbol
27     entrada.estado = 'usado'
28     entrada.save()
29     set_usados.agregar(codigo)
30     arbol_codigos.insertar(codigo)
31
32     Validacion.objects.create(entrada=entrada, resultado='Válido')
33     return Response({'mensaje': ' Código válido. Acceso permitido'})
34
```

The image shows a code editor interface with three main panels:

- File Explorer (Left):** Displays the project structure. Key folders include 'frontend' and 'backend'. The 'frontend' folder contains 'src' which has 'components', 'hooks', 'pages', and 'services'. The 'backend' folder contains 'api.js', 'App.jsx', 'index.css', 'main.jsx', and '.env'.
- Main Editor (Center):** Shows the code for 'ReservarAsientos.jsx'. The code uses React hooks like 'useState', 'useEffect', and 'useNavigate'. It defines state for 'searchParams', 'events', 'selectedEvent', 'selectedSection', 'selectedSeats', 'cart', 'timeLeft', 'timerActive', 'userId', 'seatMaps', and 'loadingSeats'. It includes a 'useReservations()' hook and a 'loadEvents()' function that fetches data from an API.
- Terminal (Bottom):** Shows the command 'python app.py' being executed. The output includes a warning: 'WARNING: This is a development server. Do not use it in a production deployment. Use production WSGI server instead.' and the URL 'Running on http://127.0.0.1:5002'.

Estructuras de Datos Básicas en app.py

Clase SeatMatrix

Representa la matriz de asientos de una zona dentro de un evento.

Cada asiento tiene un estado (backend/data/events):

'A' → Disponible

'H' → Reservado temporalmente

'S' → Vendido

Métodos principales:

is_available(r, c) → Verifica si un asiento está libre.

set_state(r, c, state) → Cambia el estado de un asiento.

to_serializable() → Convierte la matriz a formato JSON.

Función: Gestiona visual y lógicamente la disponibilidad de asientos por zona.

Clase CircularQueue

Implementa una cola circular FIFO con tamaño fijo (MAX_ACTIVE_RESERVATIONS).

Se usa para controlar las reservas activas en memoria (las que están “en espera”).

Métodos importantes:

push(item) → Agrega una reserva.

pop() → Saca la más antigua.

remove(item) → Elimina una reserva específica.

peek_all() → Devuelve todas las reservas activas.

Función: Evita saturar el sistema limitando la cantidad de reservas activas.

Persistencia y Archivos JSON

El sistema guarda toda la información en disco: (Hasta realizar la integracion de modulos)

Eventos: data/events/<event_id>.json

Usuarios: data/users.json

Reservas: data/reservations.json

Funciones clave:

save_json_file() y load_json_file() → Guardan o cargan datos JSON.

save_all_events_to_disk() → Persiste todos los eventos.

load_events_from_disk() → Restaura los eventos y sus zonas.

persist_users() / persist_reservations() → Guardan usuarios y reservas.

Función: Permitir que los datos se mantengan entre ejecuciones sin usar una base de datos.

Lógica de Negocio

Incluye las reglas y restricciones del sistema:

Eventos

- Se cargan desde disco o se crean de ejemplo (create_demo_events_if_missing()).
- Cada evento tiene varias zonas, cada una con su matriz de asientos y precio.

Usuarios

Se crean con nombre, DNI, email y teléfono.

Se guardan en users.json.

Reservas

Estados posibles:

'held' → temporal (reserva en espera)

'sold' → confirmada

'cancelled' → anulada

'expired' → vencida automáticamente

Restricciones:

Un usuario solo puede tener hasta 6 entradas activas.

Las reservas expiran después de 5 minutos (300 segundos).

Funciones relevantes:

cleanup_expired_reservations() → Libera los asientos expirados.

user_active_ticket_count() → Cuenta reservas activas por usuario.

Función: Garantizar la integridad y disponibilidad de los asientos, controlando tiempo y cantidad.

API REST (Flask)

La API define varios endpoints para interactuar con el sistema.

Principales rutas

Endpoint	Método	Función principal
/	GET	Verifica si la API está activa y funcionando correctamente.
/events	GET	Lista todos los eventos disponibles junto con sus zonas y precios.
/events/<event_id>/zones/<zone_id>/seats	GET	Devuelve la matriz de asientos de una zona específica.
/users	POST	Crea un nuevo usuario con nombre, DNI, email y teléfono.
/users	GET	Muestra todos los usuarios registrados en el sistema.
/reserve	POST	Crea una reserva temporal de asientos (estado “held”).
/reservation/<reservation_id>/confirm	POST	Confirma una reserva (marca los asientos como vendidos).

/reservation/<rid>/cancel	POST	Cancela una reserva en espera y libera los asientos.
/reservaciones/active	GET	Muestra todas las reservas activas en la cola circular.
/users/<user_id>/purchases	GET	Lista las compras confirmadas (“sold”) de un usuario.
/admin/reset	POST	Restaura todos los datos (modo desarrollo / reinicio del sistema).

Función: Proporcionar una interfaz HTTP para que clientes web o móviles gestionen usuarios, reservas y compras.

LandingPage.jsx



CONCIERTO X - ARENA MONUMENTAL



Estadio Nacional
Lima, Perú



Sábado, 15 de Noviembre 2025



20:00 hrs

MAPA REFERENCIAL

ESCENARIO

PLATINIUM

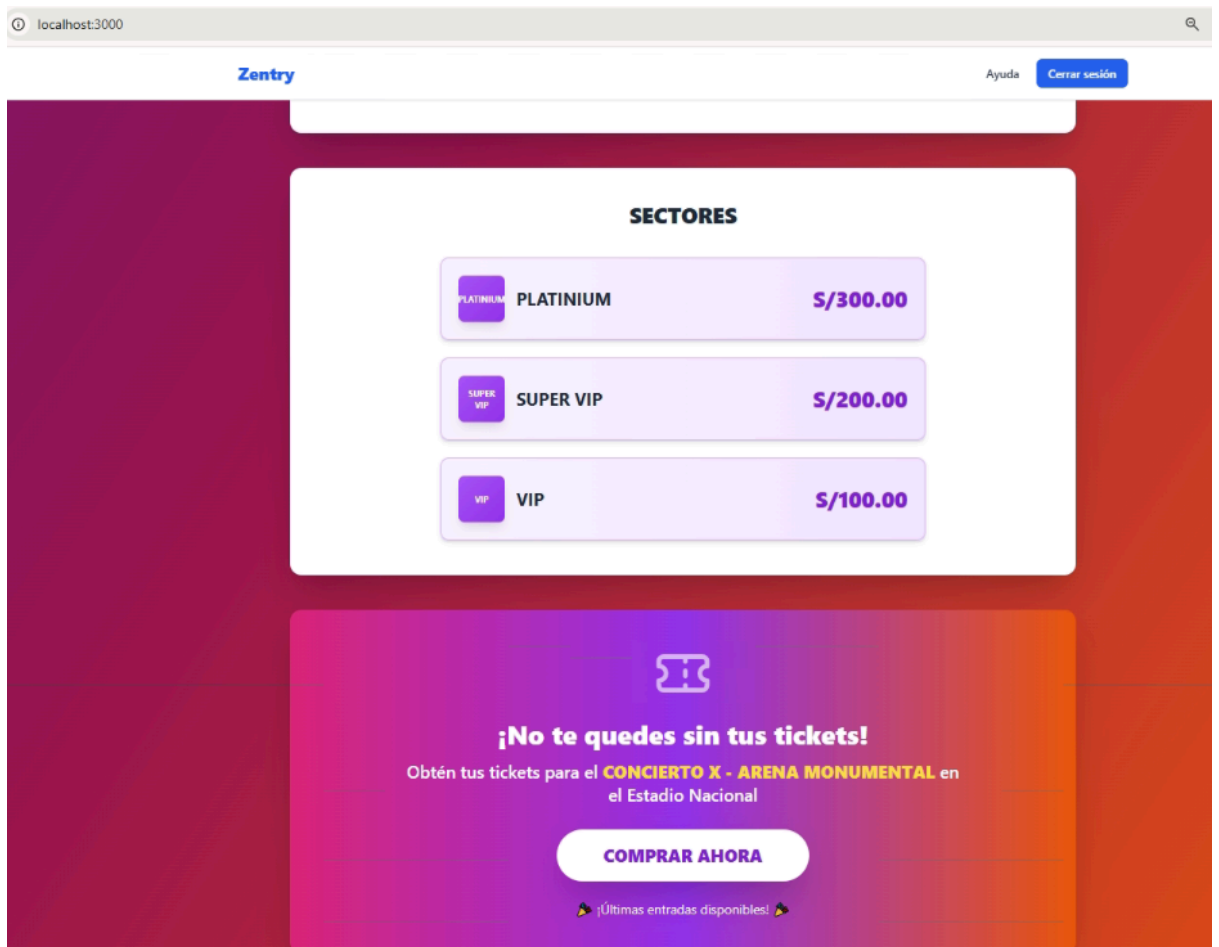
S/300.00

SUPER VIP

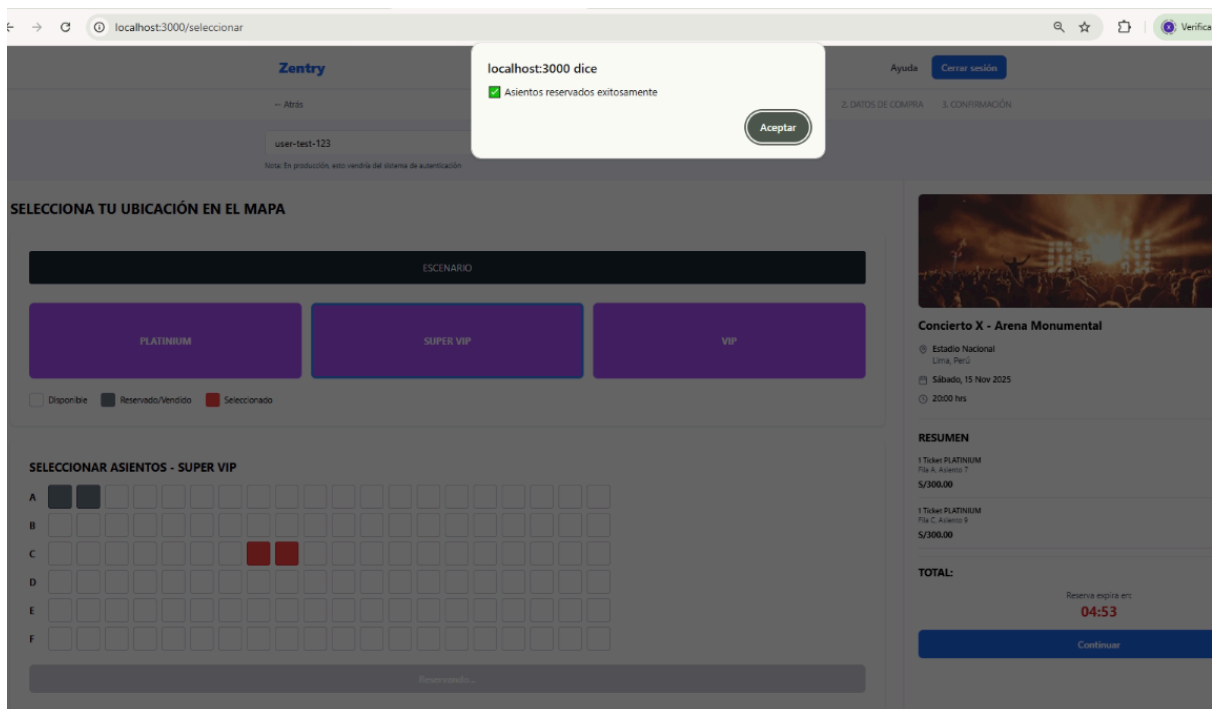
S/200.00

VIP

S/100.00



ReservarAsientos.jsx



Modulo: Carrito

1. Núcleo de Datos (Excels en /storage)

Se crearon tres archivos Excel que funcionan como la base de datos inicial:

Archivo	Propósito	Hojas y Campos
events.xlsx	Define los eventos, zonas, precios y stock.	Events(id,name,date,venue), Zones(event_id,zone_id,capacity), Prices(event_id,zone_id,stage,price), Stock(event_id,zone_id,available)
reservations.xlsx	Maneja las reservas temporales de entradas.	Reservations(reservation_id,event_id,zone_id,qty,user_id,created_at,expiry_at,status)
carts.xlsx	Guarda los ítems del carrito.	Carts(user_id,event_id,zone_id,qty,unit_price,reservation_id,updated_at)

Validaciones:

- Campos stage y status con listas desplegables (preventa, regular, ultima / pending, confirmed, released).
- Claves únicas: (event_id, zone_id) en Zones y Stock.
- Fechas en formato ISO 8601, fáciles de parsear con pandas.

2. Estructura de datos — Stack

Se implementó una pila LIFO (Last In First Out) completamente probada.

Permite manejar operaciones de deshacer (undo) dentro del carrito.

The screenshot shows a code editor with the Explorer sidebar on the left displaying a project structure for 'ZENTRY'. The 'datastructures' folder is expanded, showing 'stack.py' selected. The main editor displays the code for 'stack.py', which defines 'StackUnderflow' and 'StackOverflow' exceptions, and a 'Stack' class with methods for initialization, push, pop, and peek. The 'push' method checks for a maximum length and raises 'StackOverflow' if full. The 'pop' method checks if the stack is empty and raises 'StackUnderflow' if so. The 'peek' method checks if the stack is empty. The bottom panel shows the 'TERMINAL' output of a pytest command, indicating that 7 tests passed successfully in 2.80s.

```
datastructures > stack.py > ...
1 class StackUnderflow(Exception): ...
2 class StackOverflow(Exception): ...
3
4 class Stack:
5     def __init__(self, maxlen=None):
6         self._data = []
7         self._maxlen = maxlen
8
9     def push(self, item):
10        if self._maxlen is not None and len(self._data) >= self
11            raise StackOverflow("stack is full")
12        self._data.append(item)
13
14    def pop(self):
15        if not self._data:
16            raise StackUnderflow("stack is empty")
17        return self._data.pop()
18
19    def peek(self):
20        if not self._data:
```

PROBLEMS OUTPUT TERMINAL ...

C:\Users\Katherine\Zentry>pytest -q

..... [100%]

7 passed in 2.80s

Pruebas ejecutadas

- push/pop/peek funcionan correctamente.
- Lanza excepción StackUnderflow y StackOverflow según el caso.
- Todas las pruebas unitarias (pytest) pasaron en verde.

3. Modelo de dominio

Se definieron las entidades base que usarán los servicios:

```

domain > models.py > ...
1  from dataclasses import dataclass
2
3  class StockError(Exception): ...
4  class HoldExpired(Exception): ...
5  class LimitExceeded(Exception): ...
6
7  @dataclass(frozen=True)
8  class CartItem:
9      event_id: int
10     zone_id: str
11     qty: int
12     unit_price: float
13     reservation_id: str | None = None
14
15  @dataclass
16  class Cart:
17      user_id: str
18      items: list[CartItem]
19
20      def total_qty(self):
21          return sum(i.qty for i in self.items)
22
23      def total_amount(self):
24          return sum(i.qty * i.unit_price for i in self.items)
25

```

Excepciones personalizadas:

StockError — sin stock disponible.

HoldExpired — reserva vencida.

LimitExceeded — límite de compra superado.

4. Repositorios de prueba en excel

a) EventsExcelRepo

Encargado de leer precios y stock directamente desde el Excel.

```
repositories > excel > events_excel_repo.py > ...
1  from pathlib import Path
2  import pandas as pd
3
4  class EventsExcelRepo:
5      def __init__(self, storage_dir: str | Path = "storage"):
6          self.events_xlsx = Path(storage_dir) / "events.xlsx"
7
8      def _read(self, sheet: str) -> pd.DataFrame:
9          return pd.read_excel(self.events_xlsx, sheet_name=sheet, dtype=str)
10
11      def get_zone_stock(self, event_id: int, zone_id: str) -> int:
12          df = self._read("Stock")
13          row = df[(df["event_id"] == str(event_id)) & (df["zone_id"] == zone_id)]
14          if row.empty:
15              raise KeyError(f"Stock not found for ({event_id},{zone_id})")
16          return int(row.iloc[0]["available"])
17
18      def get_zone_price(self, event_id: int, zone_id: str, stage: str | None = None) -> float:
19          df = self._read("Prices")
20          sel = df[(df["event_id"] == str(event_id)) & (df["zone_id"] == zone_id)]
21          if stage:
22              sel = sel[sel["stage"] == stage]
23          if sel.empty and not stage:
24              pref = df[(df["event_id"] == str(event_id)) & (df["zone_id"] == zone_id) & (df["stage"] == "regular")]
25              sel = pref if not pref.empty else df[(df["event_id"] == str(event_id)) & (df["zone_id"] == zone_id) & (df["stage"] == "preventa")]
26          if sel.empty:
27              raise KeyError(f"Price not found for ({event_id},{zone_id},{stage})")
28          return float(sel.iloc[0]["price"])
29
30      __all__ = ["EventsExcelRepo"]
31
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
7 passed in 2.88s
C:\Users\Katherine\Zentry>
```

b) ReservationsExcelRepo

Controla las reservas temporales y su expiración.

```
repositories > excel > reservations_excel_repo.py > ...
1  from pathlib import Path
2  from datetime import datetime, timedelta, timezone
3  import pandas as pd
4
5  ISO = "%Y-%m-%dT%H:%M:%S%z"
6
7  class ReservationsExcelRepo:
8      def __init__(self, storage_dir: str | Path = "storage"):
9          self.path = Path(storage_dir) / "reservations.xlsx"
10         self.sheet = "Reservations"
11
12     def _read(self) -> pd.DataFrame:
13         return pd.read_excel(self.path, sheet_name=self.sheet, dtype=str)
14
15     def _write(self, df: pd.DataFrame):
16         with pd.ExcelWriter(self.path, engine="openpyxl", mode="w") as w:
17             df.to_excel(w, sheet_name=self.sheet, index=False)
18
19     def hold(self, reservation_id: str, event_id: int, zone_id: str, qty: int, user_id: str, ttl_minutes: int):
20         now = datetime.now(timezone.utc).astimezone()
21         created_at = now.strftime(ISO)
22         expiry_at = (now + timedelta(minutes=ttl_minutes)).strftime(ISO)
23         df = self._read()
24         new_row = {
25             "reservation_id": reservation_id,
26             "event_id": str(event_id),
27             "zone_id": zone_id,
28             "qty": str(qty),
29             "user_id": user_id,
30             "created_at": created_at,
31             "expiry_at": expiry_at,
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
C:\Users\Katherine\Zentry>pytest -q
..... [100%]
7 passed in 2.80s

C:\Users\Katherine\Zentry>
```

Pruebas realizadas:

- Inserción de reserva (hold) → fila con status="pending".
- Cambio de estado (release, confirm).
- Verificación de vencimiento (is_expired).
- Casos de error (KeyError en reserva inexistente).
- Todo validado con pytest.

5. Conclusiones y proximos pasos

Conclusiones

- El núcleo de datos quedó estable, reutilizable y probado.
- Se definieron correctamente las interfaces de servicios, asegurando modularidad.
- La pila LIFO garantiza que el Undo del carrito pueda implementarse de forma segura.
- Los Excel funcionan como persistencia temporal clara y validada.

Próximos pasos

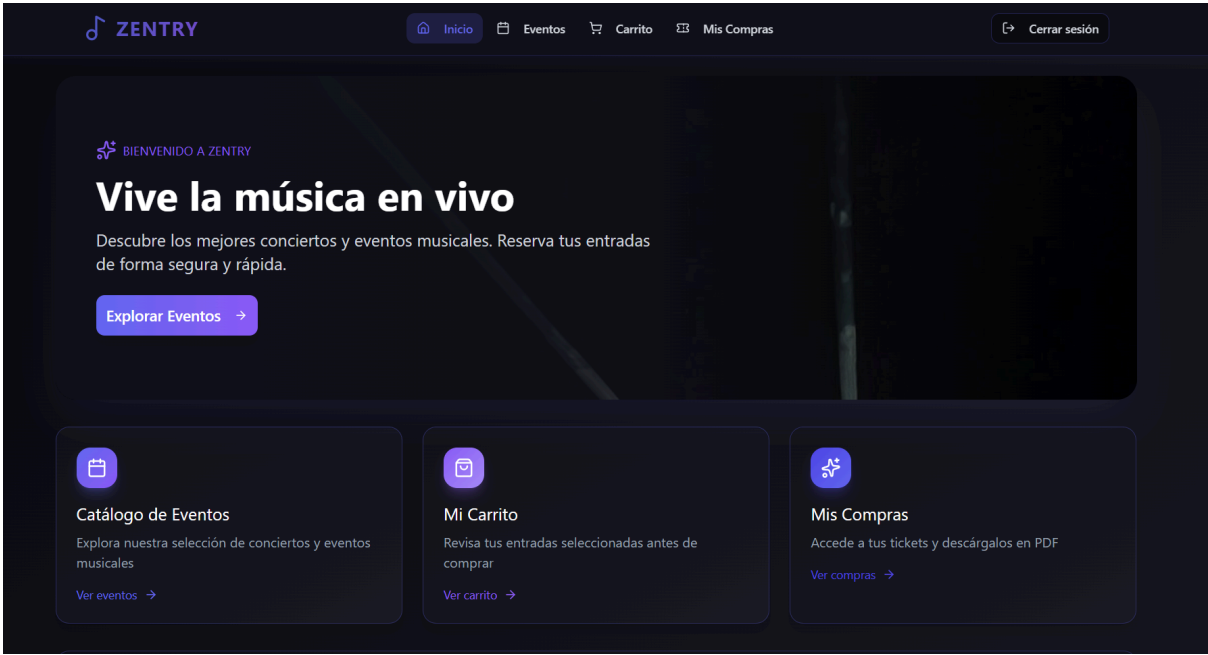
- Implementar CartService con operaciones add_item, remove_item, update_qty, undo, get_cart.
- Integrar con ReservationService y EventsExcelRepo.
- Añadir casos de prueba de integración (test_cart_undo.py, test_cart_integration.py).
- Mantener reservas TTL y validación de stock.

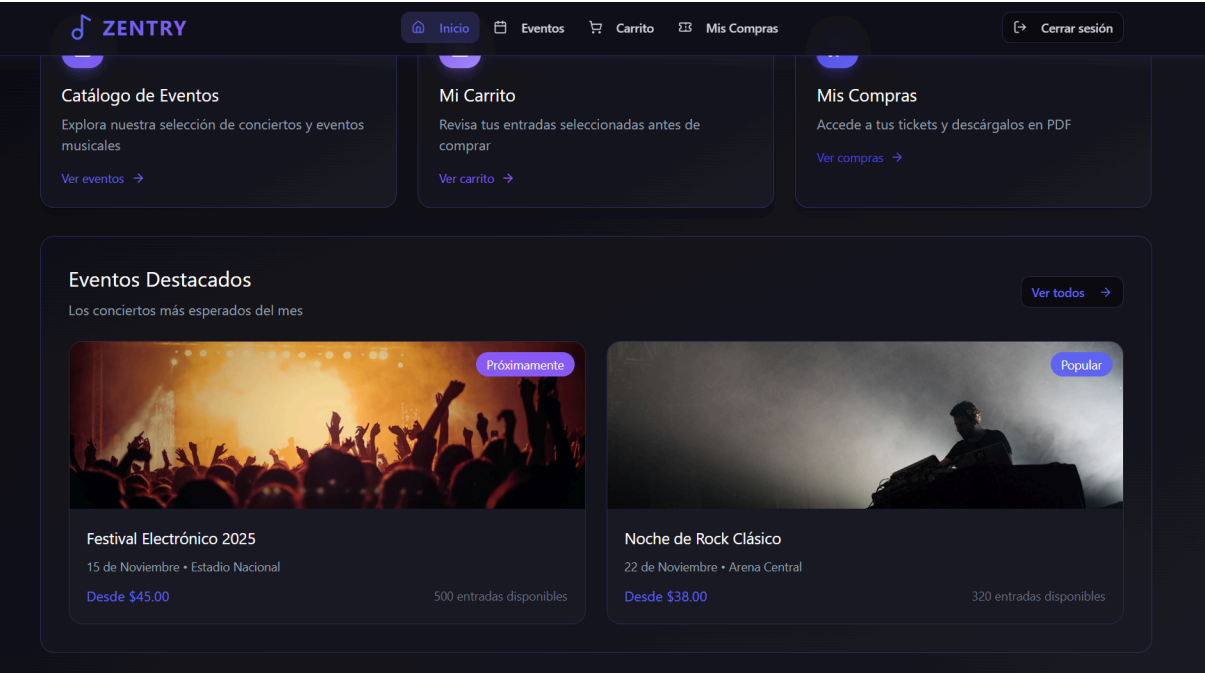
d. Interfaz inicial: pantalla principal y menú de opciones

Login/Inicio de Sesión: Tanto como para **cliente** o **staff**.

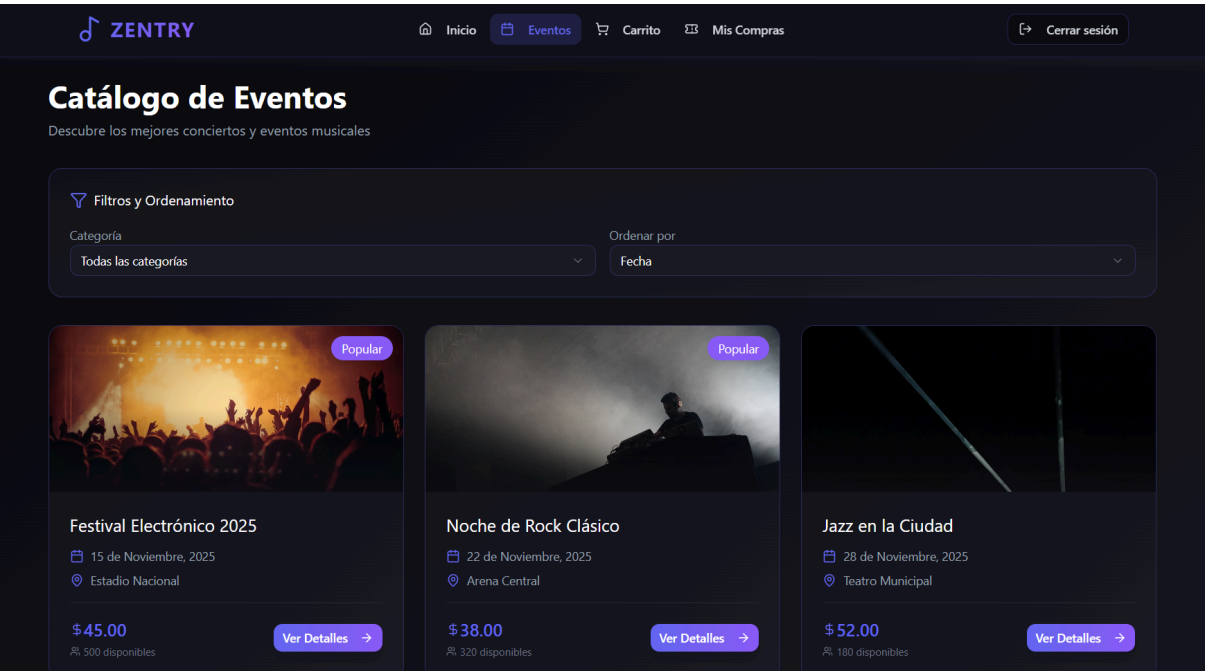


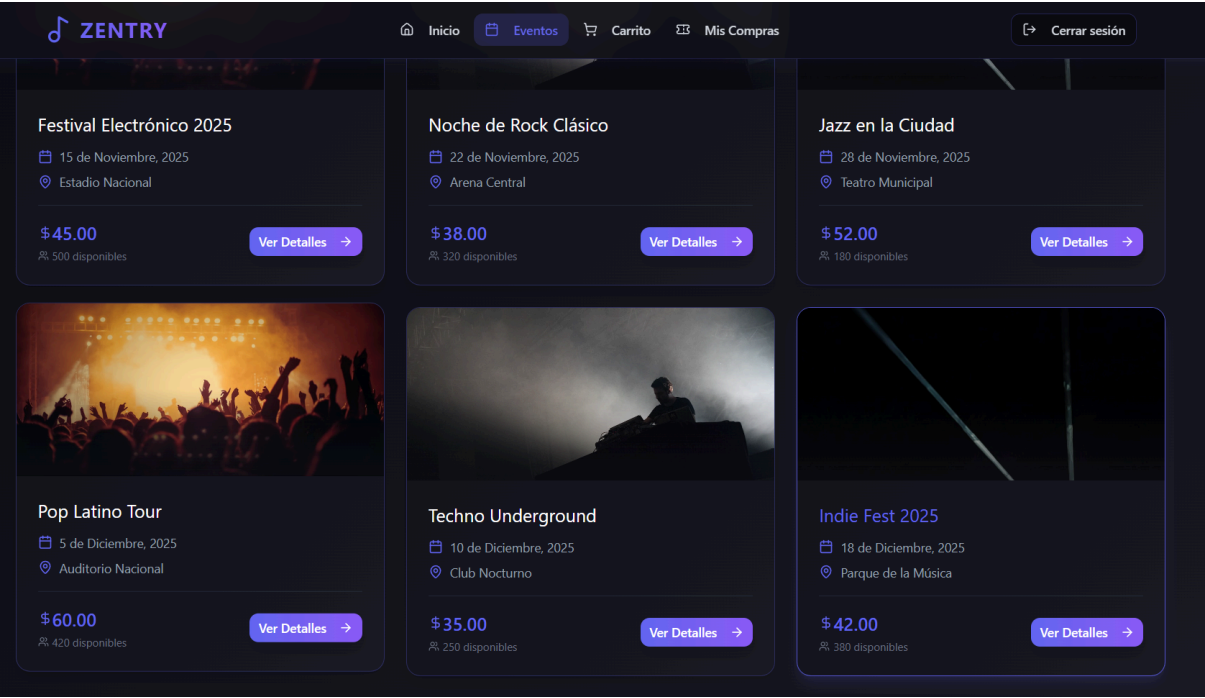
Menú de Inicio para Cliente:



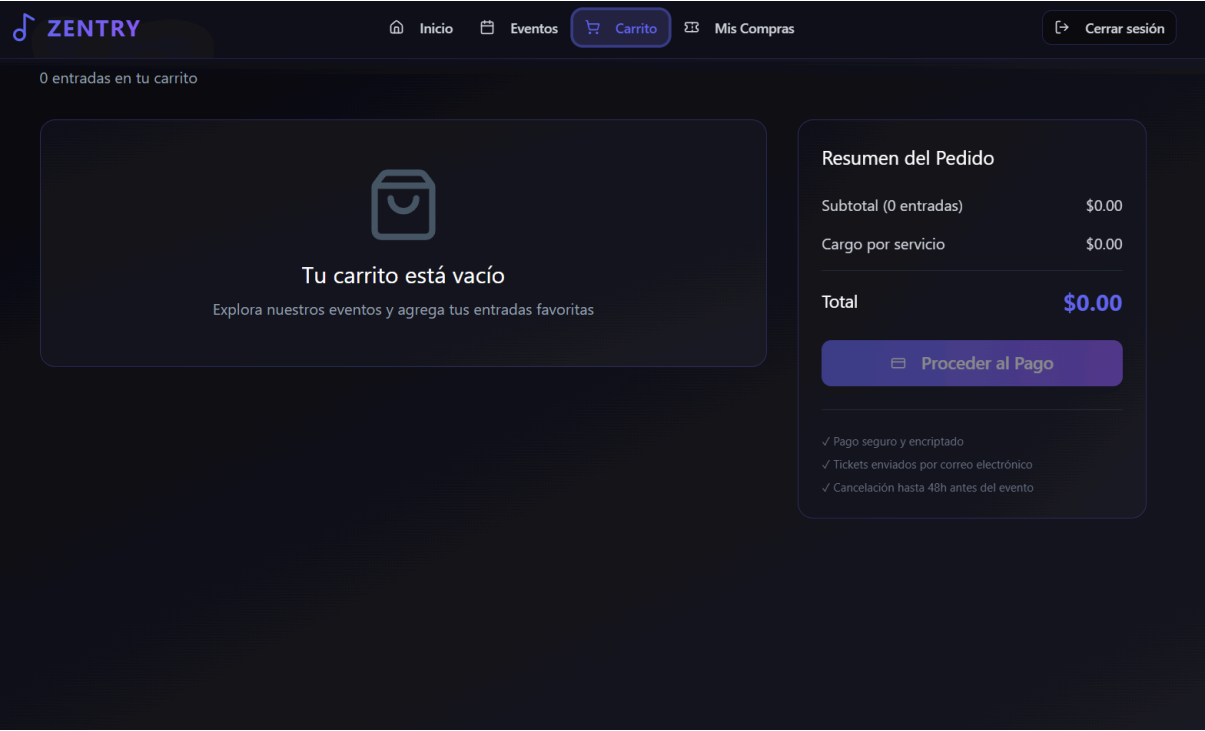


Menú de Eventos:

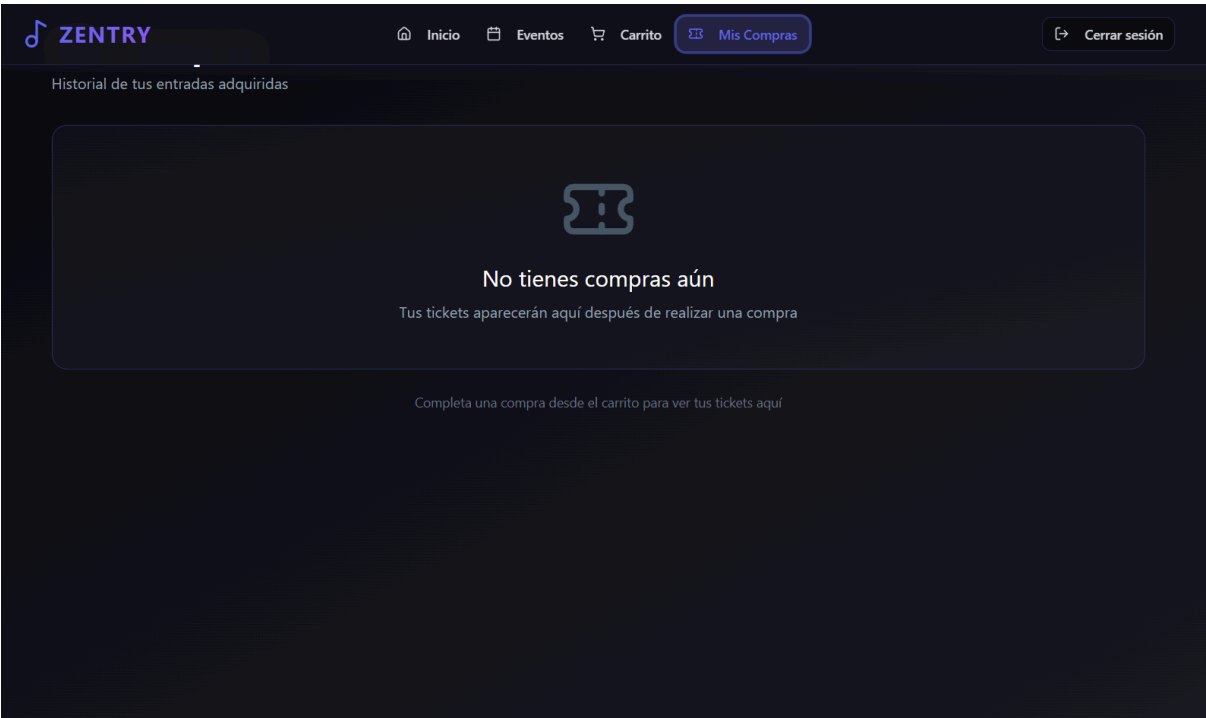




Menú Carrito:



Menú Mis Compras:



Menú para el Staff:

